September 24, 2025

5DV243 Artificial Intelligence

Assignment 1 – Othello

Sérgio Apolinário da Costa ***(mai25sca)***
Josepovits Gábor ***(ens25jgr)***

Graders
Filip Naudot, Timotheus Kampik, Igor Ryazanov

1. Introduction

This project implements an **Othello Reversi game engine** and uses **Iterative Deepening Search** (IDS) combined with **Alpha-Beta pruning**. The main goal is to find the best possible move within a given time limit and a game board. The IDS ensures that the program always returns a valid move, even if the search is interrupted, while Alpha-Beta pruning makes the search more efficient by eliminating unnecessary branches. An evaluation function is also applied to score a board position and help to make the decision.

2. Time control and IDS

A key requirement of the solution is to search as deeply as possible without exceeding the given time limit. To achieve this, the program uses Iterative Deepening Search (IDS). IDS works by repeatedly running a depth-limited Alpha-Beta search, starting from depth 1 and increasing the depth by one after each successful search. This way, the algorithm always has a valid best move from the deepest fully completed search, even if time runs out before the next depth is finished.

To handle timing, the program wraps each Alpha-Beta search inside a separate thread controlled by a Future and a timeout. If the search at the current depth does not complete within the remaining time, the search is canceled, and the last valid result is used. This ensures that the program never exceeds the time limit. Additionally, the algorithm itself checks for interruptions during search using *Thread.interrupted()*. Whenever this signal is detected, a *TimeUpException* is thrown. This exception acts as a safeguard, immediately halting the search when time runs out.

3. Heuristics

We implemented and tested several heuristics to improve the evaluation of board positions during Alpha-Beta search. Each heuristic extends the **Plain** version.

The **plain** solution is the *CornerSideEvaluator.java* file. This baseline heuristic uses a fixed scoring system that prioritizes corners as the most valuable squares, followed by sides and then regular pieces. The static weights used are:

- *cornerMultiplier = 1000*
- *sideMultiplier = 10*
- *pieceMultiplier =* 1

This creates a simple but effective evaluation function, where controlling corners heavily influences the final score.

The **ordered moves** solution is the *AlphaBetaOrderingMoves.java* file. Unlike the previous heuristics, this approach does not change the scoring formula. Instead, it reorders the search tree expansion in the Alpha-Beta algorithm. It moves that place discs on corners are explored first, followed by sides, and then remaining squares. This can improve pruning efficiency and search depth, particularly with short time limits.

The **danger squares** solution is the *CornerSideDangerSquaresEvaluator.java* file. This heuristic extends the Plain approach but adds a penalty for dangerous moves. Any square adjacent to a corner, which can potentially allow the opponent to capture that corner, is assigned a negative weight of *-10*. This discourages risky placements near corners, prioritizing positional safety over immediate gain.

The **variable weights** solution is the *CornerSideVariableWeightsEvaluator.java* file. This approach introduces dynamic scoring based on the phase of the game, adapting the importance of each element as the board evolves:

- Early game *(fewer than 20 pieces, focuses on strategic positioning and long-term control)*:
  - *cornerMultiplier = 1500*
  - *sideMultiplier = 20*
  - *pieceMultiplier = 1*
- Midgame *(20–50 pieces, balances between piece advantage and positional strength)*:
  - *cornerMultiplier = 800*
  - *sideMultiplier = 10*
  - *pieceMultiplier = 5*
- Endgame *(more than 50 pieces, prioritizes maximizing disc count as the game approaches its conclusion)*:
  - *cornerMultiplier = 300*
  - *sideMultiplier = 5*
  - *pieceMultiplier = 100*

4. The make move algorithm

The *makeMove()* function is responsible for applying a player's move to the current Othello board and returning a new *OthelloPosition* representing the resulting state. It first checks for thread interruption to respect the time limit, throwing a *TimeUpException* if needed. If the move is a **pass**, it simply toggles the active player and returns a cloned board. Otherwise, it places the player's disc at the specified row and column and then calls *flipDiscs* in all eight directions to capture the opponent's pieces according to Othello rules. Finally, it switches the active player and returns the new position, allowing the search algorithm to simulate moves without modifying the original board.

5. Testing results

To evaluate the effectiveness of the implemented heuristics, we tested the AI against a naive Othello player under different time limits. Each test compares how variations of the AI's evaluation function perform, including base behavior, move ordering, penalizing risky squares near corners, and adapting weights according to the game stage. The results show how search time and heuristic design affect the AI's ability to make strong moves and win against a simple opponent.

	Plain		Ordered moves		Danger squares		Variable weights	
Player	*White*	*Black*	*White*	*Black*	*White*	*Black*	*White*	*Black*
2 sec.	*38*	*10*	*24*	*51*	*44*	*10*	*14*	*40*
3 sec.	*48*	*24*	*36*	*34*	*46*	*10*	*46*	*34*
4 sec.	*43*	*24*	*26*	*12*	*28*	*10*	*42*	*28*

Our AI won *Our AI lost*

The **plain** solution is a solid baseline, especially as white. As black, it struggles but benefits from longer search times.

The **ordered moves** solution, provides the best short-time performance, especially as black at 2 seconds. However, at 4 seconds performance collapses, suggesting that ordering harms deeper searches.

The **danger squares** solution works reasonably at 2-3 seconds as white, but performance drops sharply at 4 seconds. As black, it is consistently weak, probably because the penalty is too strict and reduces flexibility.

The **variable weights** solution is the most consistent and robust solution, especially for black. It starts weak at 2 seconds as white but improves significantly with more time, confirming that dynamic weights benefit from deeper searches.

The variable weights heuristic offers the best balance and long-term consistency, particularly for black and longer time limits. Ordered Moves is strong in fast games, while danger squares prove too restrictive. The plain heuristic remains competitive, showing that Alpha-Beta search alone already provides a solid foundation. The final heuristics we choose was the **variable weights**.

6. Conclusion

During the development of this Othello game engine, we faced several challenges. One of the main difficulties was implementing effective heuristics that would evaluate board positions accurately at different stages of the game. Determining appropriate weights for pieces, corners, and sides was also a challenge so that we could make strategic decisions without overvaluing any factor. Another major challenge was handling the time limit correctly while implementing Iterative Deepening Search, ensuring the program could explore as deeply as possible within the allowed time, while stopping safely if the time expired.

We worked together and helped each other throughout the development process, but mainly each of us focused on different areas to ensure progress and integration of the components into a fully functioning engine. Sérgio Costa focused more on constructing the time-limited IDS framework, implementing Alpha-Beta pruning, and developing the *makeMove* function for simulating board states. Josepovits Gábor concentrated on designing the heuristics and achieving a balanced AI behavior.

Each of us also created several different heuristics, which we tested against each other to evaluate their effectiveness and refine the AI's strategy.